

BRAC's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 1

Roll no: 28 PRN No: 12411933 Name of Student: **Pragati S. Rakhunde**

Subject Name : Deep Learning

Problem Statement :

Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test split, and visualization on a sample dataset.

Algorithm :

Fashion MNIST Dataset Classification Using TensorFlow/Keras

1. Start the program.
2. Import the required libraries: TensorFlow, Keras, NumPy, Pandas, and Matplotlib.
3. Download and load the Fashion MNIST dataset (training and testing data).
4. Read the dataset and separate the features (pixel values) and labels (class names).
5. Reshape the image data from 784 pixels to 28×28 grayscale images.
6. Visualize a few sample images along with their corresponding class labels.
7. Normalize the pixel values by dividing each pixel value by 255 so that all values lie between 0 and 1.
8. Use the provided training set (60,000 images) and testing set (10,000 images) for model development and evaluation.
9. Create a Sequential neural network model consisting of:
 1. Flatten layer
 2. Dense hidden layer with ReLU activation
 3. Dense output layer with Softmax activation
10. Compile the model using the Adam optimizer, Sparse Categorical Crossentropy loss function, and Accuracy as the evaluation metric.
11. Train the model on the training dataset for a fixed number of epochs and validate it using a validation split.

12. Evaluate the trained model using the testing dataset to obtain the test accuracy and loss.
13. Predict the class labels of the test images using the trained model.
14. Visualize the training and validation accuracy using graphs.
15. Display the predicted class and compare it with the actual class label.
16. End the program.

Code :

Step 1: Import Required Libraries

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Step 2: Load the Fashion MNIST Dataset

```
train = pd.read_csv("fashion-mnist_train.csv")
test = pd.read_csv("fashion-mnist_test.csv")
```

Step 3: Separate Features and Labels

```
X_train = train.iloc[:, 1:].values
y_train = train.iloc[:, 0].values
X_test = test.iloc[:, 1:].values
y_test = test.iloc[:, 0].values
```

Step 4: Reshape the Images

```
X_train = X_train.reshape(-1, 28, 28)
X_test = X_test.reshape(-1, 28, 28)
```

Step 5: Normalize the Data

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Step 6: Display Sample Images

```
class_names = ["T-shirt", "Trouser", "Pullover", "Dress", "Coat", "Sandal", "Shirt", "Sneaker",  
"Bag", "Ankle Boot"]  
plt.figure(figsize=(10,5))  
for i in range(10):  
    plt.subplot(2,5,i+1)  
    plt.imshow(X_train[i], cmap='gray')  
    plt.title(class_names[y_train[i]])  
    plt.axis('off')  
plt.show()
```

Step 7: Build the Neural Network Model

```
model = keras.Sequential([  
    keras.layers.Flatten(input_shape=(28,28)),  
    keras.layers.Dense(128, activation='relu'),  
    keras.layers.Dense(10, activation='softmax')  
])
```

Step 8: Compile the Model

```
model.compile( optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'] )
```

Step 10: Evaluate the Model

```
loss, accuracy = model.evaluate(X_test, y_test)  
print("Test Loss:", loss)  
print("Test Accuracy:", accuracy)
```

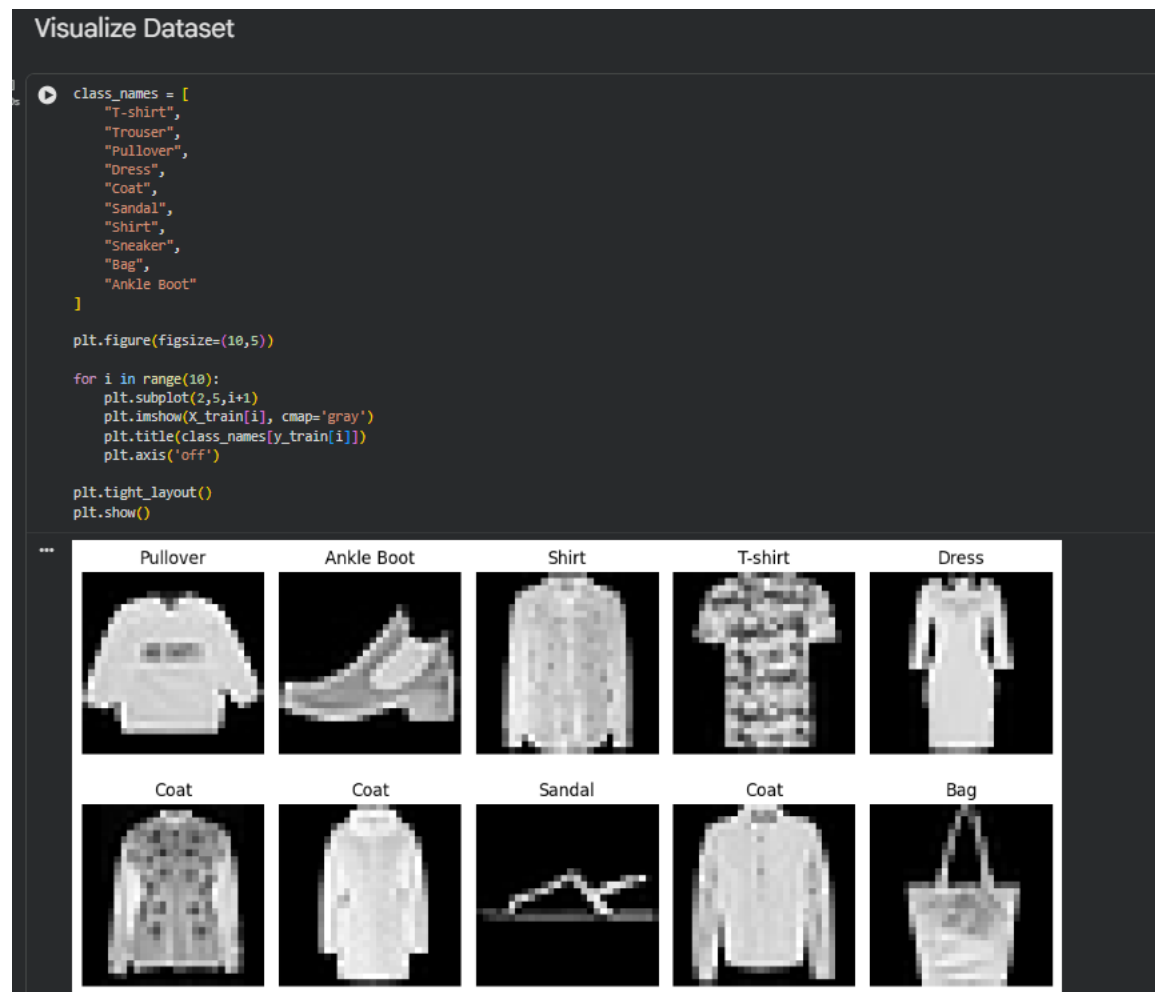
Step 11: Predict the Class of a Test Image

```
prediction = model.predict(X_test)  
print("Predicted Class:", class_names[np.argmax(prediction[0])])  
print("Actual Class:", class_names[y_test[0]])
```

Step 12: Plot Training and Validation Accuracy

```
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy') plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training vs Validation Accuracy")
plt.legend()
plt.show()
```

Output :



Train Model

```
[24] ✓ 36s history = model.fit(  
    X_train,  
    y_train,  
    epochs=5,  
    validation_split=0.2  
)
```

```
... Epoch 1/5  
1500/1500 ————— 7s 4ms/step - accuracy: 0.8151 - loss: 0.5278 - val_accuracy: 0.8393 - val_loss: 0.4564  
Epoch 2/5  
1500/1500 ————— 8s 5ms/step - accuracy: 0.8565 - loss: 0.3989 - val_accuracy: 0.8574 - val_loss: 0.4007  
Epoch 3/5  
1500/1500 ————— 6s 4ms/step - accuracy: 0.8709 - loss: 0.3573 - val_accuracy: 0.8662 - val_loss: 0.3687  
Epoch 4/5  
1500/1500 ————— 8s 5ms/step - accuracy: 0.8818 - loss: 0.3281 - val_accuracy: 0.8760 - val_loss: 0.3601  
Epoch 5/5  
1500/1500 ————— 7s 4ms/step - accuracy: 0.8876 - loss: 0.3097 - val_accuracy: 0.8802 - val_loss: 0.3440
```

Evaluate Model

```
[25] ✓ 1s loss, accuracy = model.evaluate(X_test,y_test)  
  
print("Loss :",loss)  
print("Accuracy :",accuracy)
```

```
... 313/313 ————— 1s 3ms/step - accuracy: 0.8790 - loss: 0.3353  
Loss : 0.335264563566048584  
Accuracy : 0.8790000081062317
```

Prediction

```
[26] ✓ 1s pred = model.predict(X_test)  
  
print("Predicted:",class_names[np.argmax(pred[0])])  
print("Actual:",class_names[y_test[0]])  
  
plt.imshow(X_test[0], cmap='gray')  
plt.axis('off')  
plt.show()
```

```
... 313/313 ————— 1s 3ms/step  
Predicted: T-shirt  
Actual: T-shirt
```



Accuracy Visualization

[27]
✓ 0s

```
plt.figure(figsize=(8,5))  
  
plt.plot(history.history['accuracy'], label='Training Accuracy')  
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')  
  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.title("Training vs Validation Accuracy")  
plt.legend()  
plt.show()
```

